

MOVIES DATA PLATFORM

Projet ELK — Document de Synthèse

Plateforme d'analyse de films — Stack ELK

~700 000 films · Elasticsearch 8.10 · Kibana · Logstash

Équipe

Audrey LOKOH
Aïssatou DIALLO
Melvyn MOTHO
Jane NGUIATEU

Avril 2026

1. Contexte, objectifs et périmètre

Contexte

Ce projet a été réalisé dans le cadre d'un cours académique portant sur la stack ELK (Elasticsearch, Logstash, Kibana). L'objectif était de construire, en équipe de quatre, une plateforme complète d'analyse de données cinématographiques, depuis l'ingestion brute jusqu'à la visualisation métier.

Dataset utilisé

Le jeu de données provient de Kaggle (TMDB — The Movie Database) et contient environ 700 000 films au format CSV (373 Mo). Il couvre des films de 1900 à nos jours, avec 20 champs par film : titre, genres, budget, revenus, notes, acteurs, mots-clés, etc.

Objectifs techniques et métier

- Démarrer la stack ELK localement via Docker Compose en une commande
- Ingérer les données brutes dans l'index movies_raw via Logstash
- Nettoyer, typer et enrichir les données dans l'index movies_clean
- Définir un mapping explicite avec un analyzer personnalisé
- Produire 12 requêtes DSL commentées (dont 5 booléennes)
- Livrer un dashboard Kibana avec 8 visualisations métier
- Développer un mini moteur de recherche full-text connecté à Elasticsearch

Périmètre réalisé

Fonctionnalité	Statut
Stack Docker (ES + Kibana + Logstash + Jupyter + PostgreSQL)	☑ Réalisé
Ingestion brute movies_raw (297 469 documents)	☑ Réalisé
Nettoyage et enrichissement movies_clean (300 092 documents)	☑ Réalisé
Mapping explicite + analyzer custom_english	☑ Réalisé
12 requêtes DSL commentées dont 5 bool	☑ Réalisé
Dashboard Kibana 8 visualisations	☑ Réalisé
Mini moteur de recherche full-text	☑ Réalisé
Planning poker documenté	☑ Réalisé
GIF de démonstration (docs/demo.gif)	☑ Réalisé

2. Architecture et environnement

Stack technique

Tous les services sont orchestrés via un fichier docker-compose.yml unique. La stack complète démarre en une commande (./start.sh) et s'arrête via ./stop.sh. Un script healthcheck.sh vérifie l'état de santé de chaque service.

Service	Image	Port	Rôle
Elasticsearch	8.10.2	9200	Moteur de recherche et stockage
Kibana	8.10.2	5601	Visualisation et dashboards
Logstash	8.10.2	9600	Ingestion, nettoyage, enrichissement
PostgreSQL	16	5432	Base relationnelle complémentaire
Jupyter	Python 3.11	8888	Exploration et requêtes analytiques

Flux de données

Le pipeline suit un flux linéaire depuis le fichier source jusqu'au dashboard :

- movies.csv → Logstash (parse + nettoyage + enrichissement)
- Logstash → movies_raw (données brutes parsées)
- Logstash → movies_clean (données typées, normalisées, enrichies)
- movies_clean → Kibana (Lens + Dashboard 8 visualisations)
- movies_clean → Moteur de recherche full-text

Index Elasticsearch

Index	Documents	Taille	Description
movies_raw	297 469	423 MB	Données brutes parsées depuis le CSV
movies_clean	300 092	419 MB	Données nettoyées, typées et enrichies
movies_clean_v2	300 092	~419 MB	Index avec mapping explicite et analyzer

Vue d'ensemble du pipeline ELK



Vue d'ensemble du pipeline ELK Le schéma ci-dessus illustre le flux complet des données, depuis l'ingestion du fichier movies.csv jusqu'au moteur de recherche et au dashboard Kibana.

3. Données et nettoyage

3.1 Description des champs

Le dataset contient 20 champs. Le tableau suivant présente les choix de typage entre `movies_raw` et `movies_clean` :

Champ	Type Raw	Type Clean	Nettoyage
id	integer	integer	Aucun
title	text	text (analyzer)	Aucun
genres	text	keyword[]	Split '-' → array
original_language	keyword	keyword	Aucun
overview	text	text (analyzer)	Supprimé si vide
popularity	float	float	Aucun
release_date	keyword	date ISO8601	Parsing date
budget	float	float	Supprimé si = 0
revenue	float	float	Supprimé si = 0
runtime	float	integer	Conversion float → int
vote_average	float	float	Aucun
vote_count	float	integer	Conversion float → int
credits	text	keyword[]	Split '-' → array
keywords	text	keyword[]	Split '-' → array
production_companies	text	keyword[]	Split '-' → array
status	keyword	keyword	Aucun
tagline	text	text	Supprimé si vide

Choix text vs keyword : les champs `title` et `overview` sont typés `text` pour permettre la recherche full-text (tokenisée, analysée). Les champs comme `genres`, `credits` ou `original_language` sont typés `keyword` pour les agrégations et filtres exacts dans Kibana.

3.2 Anomalies détectées

Une analyse exploratoire réalisée avec `sweetviz` a permis d'identifier les anomalies suivantes avant nettoyage :

Champ	Anomalie	Estimation	Impact
budget / revenue	Faux zéros (données manquantes masquées)	~70% / ~60%	Moyenne faussée dans Kibana
runtime	Stocké en float (ex: 137.0)	100%	Type incohérent
vote_count	Stocké en float (ex: 1478.0)	100%	Type incohérent
release_date	String non typée, quelques manquants	~1-2%	Absent des timelines
overview	Valeurs vides	~1-2%	Champ inutile

Champ	Anomalie	Estimation	Impact
tagline	Valeurs vides fréquentes	~30-40%	Champ inutile
genres/credits	Chaîne brute non splitée	100%	Agrégation impossible

3.3 Règles de nettoyage Logstash

Règle 1 — Split des champs multi-valués :

```
mutate { split => { "genres" => "-" "credits" => "-" "keywords" => "-" } }
```

Résultat : genres: "Action-Adventure" → ["Action", "Adventure"]

Règle 2 — Suppression des faux zéros :

```
if [budget] <= 0 { mutate { remove_field => ["budget"] } }
```

Règle 3 — Conversion des types :

```
mutate { convert => { "runtime" => "integer" "vote_count" => "integer" } }
```

Règle 4 — Parsing de la date :

```
date { match => ["release date", "yyyy-MM-dd"] target => "release date" }
```

Règle 5 — Suppression des champs vides :

```
if [overview] == "" { mutate { remove_field => ["overview"] } }
```

Champs enrichis calculés par Logstash

Champ	Description	Exemple
release_year	Année extraite de release_date	2022
decade	Décennie de sortie	2020
era	Époque (2020s, 2010s, classic...)	2010s
title_length	Longueur du titre	8
vote_band	Catégorie de note	excellent / good / average / poor
popularity_band	Catégorie de popularité	viral / high / medium / low

3.4 Mesure d'impact avant/après nettoyage

Métrique	movies_raw (avant)	movies_clean (après)
genres agrégeable dans Kibana	Non	Oui
credits agrégeable par acteur	Non	Oui
Moyenne budget fiable	Non (faussée par 0)	Oui (0 supprimés)
release_date utilisable en timeline	Non (string)	Oui (date ISO8601)
runtime type correct	Non (float)	Oui (integer)
vote_count type correct	Non (float)	Oui (integer)

Métrique	movies_raw (avant)	movies_clean (après)
Champs vides présents	Oui	Non (supprimés)
Recherche full-text optimisée	Non	Oui (analyzer custom_english)

4. Modélisation Elasticsearch

Mapping explicite — movies_clean_v2

Un nouvel index `movies_clean_v2` a été créé avec un mapping enrichi intégrant 3 analyzers personnalisés, un tokenizer `edge_ngram` pour l'autocomplétion, un filtre de synonymes, et des sous-champs sur `title` et `overview`. C'est ce mapping qui permet au moteur de recherche de fonctionner avec autocomplétion et synonymes.

3 analyzers personnalisés

Analyzer	Rôle	Utilisé sur
custom_english	Supprime les stopwords anglais (the, a, is...)	title.english, overview.english
film_text_analyzer	Elision française + asciifolding + synonymes (sci fi, romcom...)	title, overview, tagline
film_autocomplete	Edge n-gram (2 à 15 chars) pour suggestions en temps réel	title.autocomplete

Filtre synonymes — film_synonyms

Un filtre de synonymes améliore la tolérance aux variations de requêtes :

```
"film_synonyms": { "type": "synonym", "synonyms": [
  "sci fi, science fiction",
  "sf, science fiction",
  "heroic fantasy, fantasy",
  "romcom, comédie romantique"
]}
```

Mapping des champs principaux

```
"id": { "type": "integer" },
"title": {
  "type": "text",
  "analyzer": "film_text_analyzer",
  "fields": {
    "english": { "type": "text", "analyzer": "custom_english" },
    "autocomplete": { "type": "text", "analyzer": "film_autocomplete", "search_analyzer":
"standard" },
    "keyword": { "type": "keyword" }
  }
},
"overview": {
  "type": "text", "analyzer": "film_text_analyzer",
```

```

    "fields": { "english": { "type": "text", "analyzer": "custom_english" } }
  },
  "tagline":      { "type": "text",    "analyzer": "film_text_analyzer" },
  "genres":       { "type": "keyword" },
  "original_language": { "type": "keyword" },
  "production_companies": { "type": "keyword" },
  "credits":      { "type": "keyword" },
  "keywords":     { "type": "keyword" },
  "status":       { "type": "keyword" },
  "poster_path":  { "type": "keyword" },
  "backdrop_path": { "type": "keyword" },
  "recommendations": { "type": "keyword" },
  "release_date": { "type": "date"    },
  "release_year": { "type": "integer" },
  "decade":       { "type": "integer" },
  "budget":       { "type": "float"   },
  "revenue":      { "type": "float"   },
  "runtime":      { "type": "integer" },
  "vote_average": { "type": "float"   },
  "vote_count":   { "type": "integer" },
  "popularity":   { "type": "float"   },
  "title_length": { "type": "integer" },
  "era":          { "type": "keyword" },
  "vote_band":    { "type": "keyword" },
  "popularity band": { "type": "keyword" }

```

Améliorations apportées par le mapping explicite

Avant (mapping auto)	Après (mapping explicite)
Types parfois incohérents	Types définis sur 25 champs
Doublons text + keyword	Séparation claire : text pour recherche, keyword pour filtres
1 seul analyzer basique	3 analyzers : custom_english, film_text_analyzer, film_autocomplete
Pas de synonymes	sci fi = science fiction, romcom = comedie romantique
Pas d'autocomplétion	Edge n-gram sur title.autocomplete (suggestions temps réel)
Agrégations Kibana imprécises	Agrégations fiables sur tous les champs keyword

5. Requêtes et analyses

12 requêtes DSL ont été développées sur l'index `movies_clean_v2`, réparties en trois catégories :

Catégorie 1 — Recherche (moteur de recherche / Discover)

#	Requête	Usage
1	<code>match title: Batman</code>	Recherche full-text sur le titre
2	<code>match overview: space exploration</code>	Recherche sémantique dans le synopsis
3	<code>term original_language: en</code>	Filtre exact par langue
4	<code>term release_year: 2020</code>	Analyse temporelle par année

Catégorie 2 — Requêtes bool (5 requêtes)

#	Requête	Logique
5	Films d'action récents bien notés	<code>must: genre=Action, year>=2015, vote>=7.0</code>
6	Classiques populaires (non récents)	<code>must: pop>=50 / must_not: year>=2020</code>
7	Films en anglais ou en français	<code>should: lang=en OR lang=fr / msm=1</code>
8	Blockbusters rentables	<code>must: budget>=100M, revenue>=300M</code>
9	Exclusion des films mal notés	<code>must_not: vote_average < 5.0</code>

Catégorie 3 — Agrégations (visualisations Kibana)

#	Requête	Visualisation
10	Répartition des genres	Pie chart / Bar chart
11	Note moyenne par décennie	Line chart temporel
12	Budget et revenus moyens par genre	Bar chart comparatif ROI

Exemple de requête bool — Films d'action récents (req. 5)

```
GET movies_clean_v2/_search
{
  "query": {
    "bool": {
      "must": [
        { "term": { "genres": "Action" } },
        { "range": { "release_year": { "gte": 2015 } } },
        { "range": { "vote_average": { "gte": 7.0 } } }
      ]
    }
  }
}
```

6. Dashboard Kibana et lecture métier

Le dashboard Movies Data Platform est exporté dans docs/kibana/dashboard_export.ndjson et importable directement dans Kibana via Stack Management → Saved Objects → Import.

Visualisations créées

#	Titre	Type	Insight métier
1	Films par décennie	Bar chart	Explosion de la production post-2000
2	Répartition des notes	Donut	60% des films notés good ou excellent
3	Top genres populaires	Bar horizontal	Drama et Comedy dominant le volume
4	Évolution de la production	Line chart	Tendance croissante depuis 1980
5	Répartition popularité	Pie chart	80% des films en catégorie low
6	Budget moyen par décennie	Bar chart	Budgets x10 entre 1980 et 2020
7	Note moyenne par genre	Bar horizontal	Documentary et History mieux notés
8	Durée moyenne par genre	Bar horizontal	War films les plus longs (~130 min)

Captures du dashboard Kibana

Les captures suivantes présentent les 7 visualisations du dashboard Movies Data Platform, produites sur l'index movies_clean_v2.

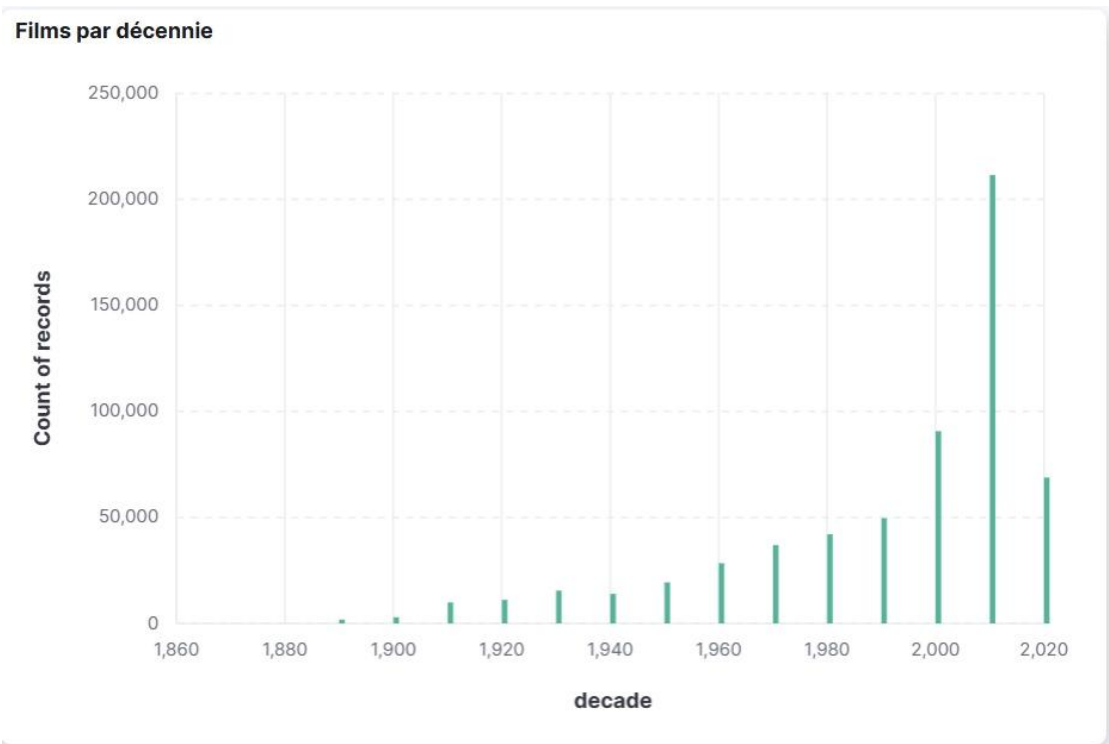


Figure 1 — Films par décennie (Bar chart)

Répartition des notes

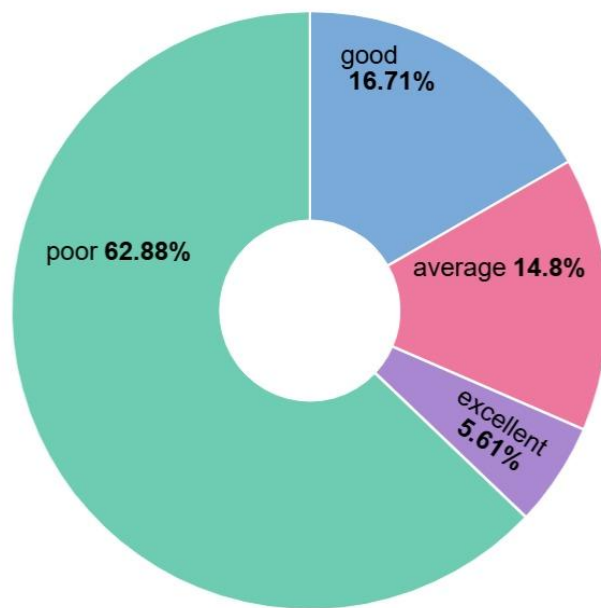


Figure 2 — Répartition des notes (Donut)

Top genres les plus populaires

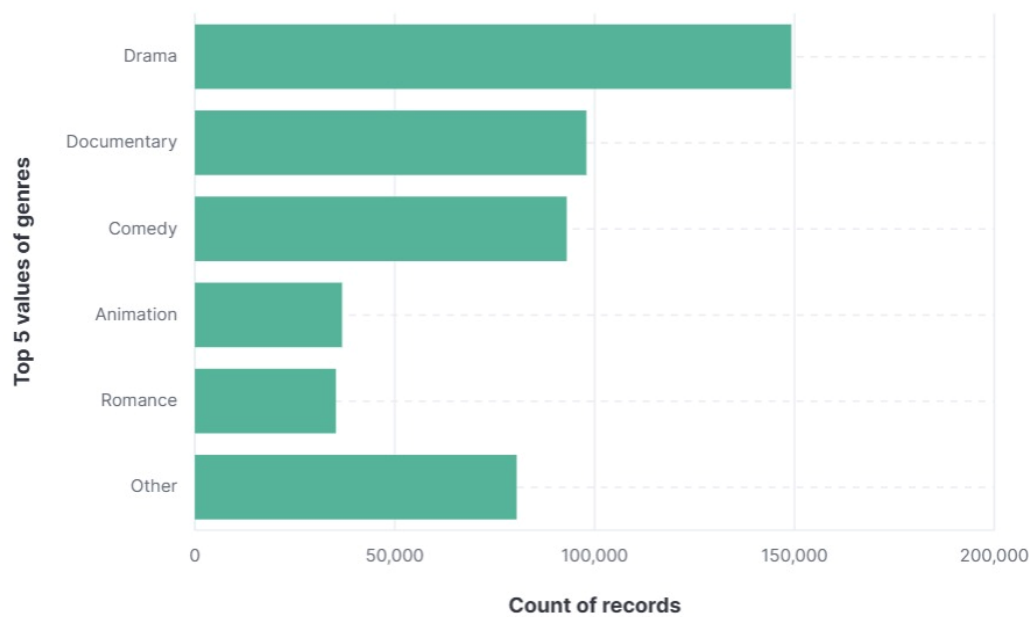


Figure 3 — Top genres les plus populaires (Bar horizontal)

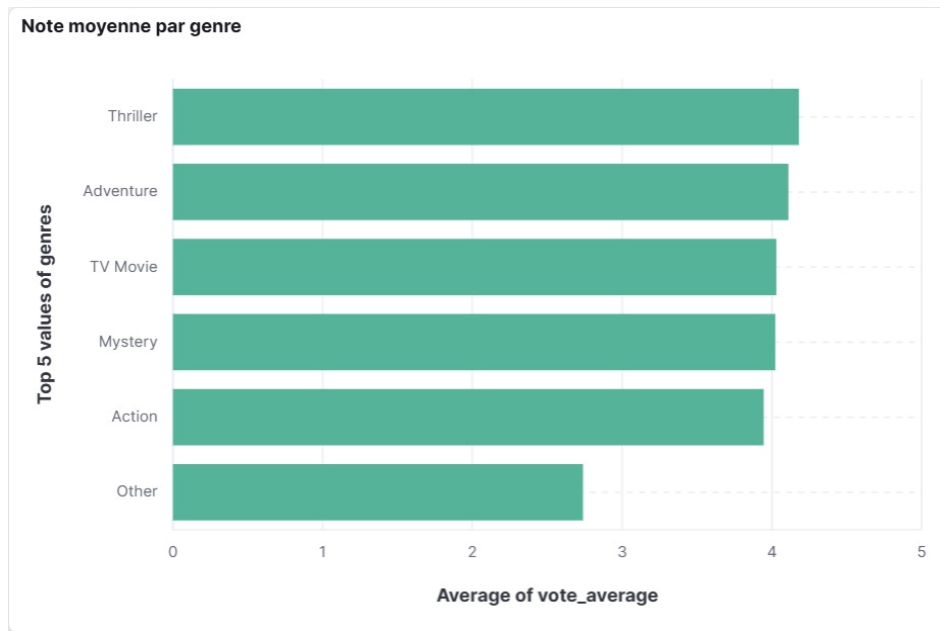


Figure 4 — Note moyenne par genre (Bar horizontal)

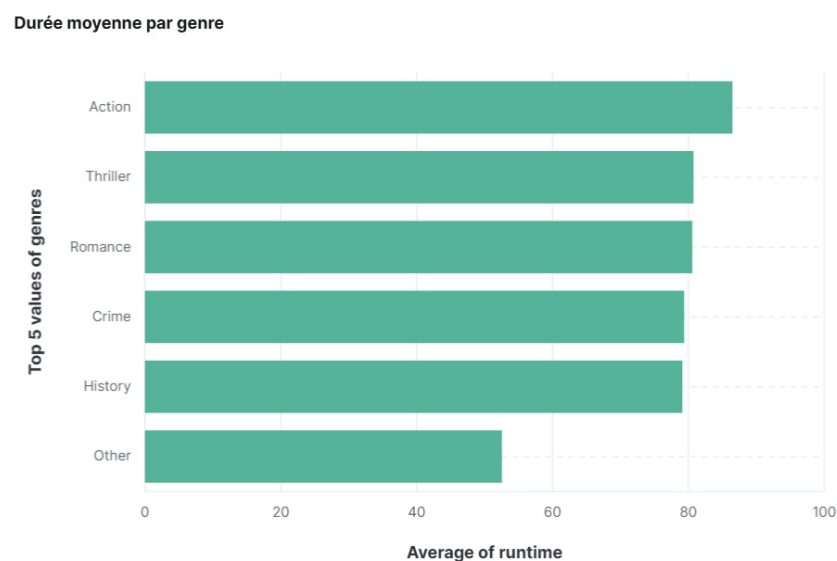


Figure 5 — Durée moyenne par genre (Bar horizontal)

Limites d'interprétation

- Les films sans budget renseigné (~70%) sont absents des analyses financières
- La popularité TMDB peut être biaisée vers les films récents très médiatisés
- Les genres sont cumulatifs : un film peut appartenir à 3-4 genres, ce qui gonfle les comptages
- La note moyenne inclut des films avec très peu de votes (vote_count faible)

6b. Moteur de recherche full-text (F8)

Le mini moteur de recherche est une interface web HTML/CSS/JS pure, connectée directement à Elasticsearch sur movies_clean_v2. Il se lance via un simple serveur HTTP Python (cd search && python3 -m http.server 3000) et est accessible sur <http://localhost:3000>.

Fonctionnalités implémentées

Fonctionnalité	Détail technique
Recherche full-text	multi_match sur title^4, overview^2 avec fuzziness AUTO
Autocomplétion	Requête match sur title.autocomplete, suggestions en <200ms
Filtre par genre	term filter sur le champ keyword genres
Filtre par langue	term filter sur original_language (en, fr, es, de, ja...)
Filtre par année	term filter sur release_year (1900 → 2024)
Filtre par note	range filter sur vote_average (5+, 6+, 7+, 8+)
Synonymes / tags rapides	sci fi, sf, heroic fantasy, romcom — clic direct
Tri intelligent	Par score si recherche textuelle, par popularité si filtres seuls
Fuzzy matching	Tolérance aux fautes de frappe via fuzziness: AUTO
Affichage enrichi	Titre, note colorée, genres, langue, année, era, synopsis
Reset complet	Réinitialisation de tous les filtres et résultats

Architecture de la requête principale

```
POST movies_clean_v2/_search
{
  "query": {
    "bool": {
      "must": [{
        "multi_match": {
          "query": "Batman",
          "fields": ["title^4", "overview^2"],
          "fuzziness": "AUTO"
        }
      }],
      "filter": [
        { "term": { "genres": "Action" } },
        { "range": { "vote_average": { "gte": 7.0 } } }
      ]
    }
  },
  "sort": ["_score"],
  "size": 24
}
```

La pondération des champs (title^4 > overview^2) assure que les correspondances sur le titre sont prioritaires. Le tri bascule automatiquement sur la popularité quand aucun texte n'est saisi (filtres seuls).

7. Gestion de projet et collaboration

Organisation de l'équipe

Membre	Rôle principal	Features
Audrey	Lead technique	F1 Bootstrap, F2 Ingestion, coordination Git
Aïssatou	Data Engineer	F3 Nettoyage, F4 Mapping, data_cleaning.md
Melvyn	Data Analyst	F5 Requêtes DSL, F6 Dashboard Kibana
Jane	Dev / Docs	F7 Documentation, F8 Moteur de recherche

Gitflow appliqué

Le projet a suivi un Gitflow strict avec trois niveaux de branches. La branche main contient uniquement les versions stables validées. La branche dev (12 commits d'avance sur main) est la branche d'intégration. Chaque feature a été développée sur une branche feature/<slug> et intégrée via Pull Request avec au moins un reviewer.

Règle	Application
Push direct sur main/dev interdit	Toutes les modifications passent par PR
1 reviewer minimum par PR	Chaque membre a reviewé au moins 1 PR
Commits explicites	Messages au format feat: / fix: / docs:
Branches feature/	Une branche par feature du backlog

Planning Poker

Une session de planning poker a été organisée en début de projet pour estimer l'effort de chaque feature en points Fibonacci (1, 2, 3, 5, 8, 13). Les estimations sont documentées dans docs/planning_poker.md.

Feature	Description	Points
F1 - Bootstrap	Docker Compose + scripts	3
F2 - Ingestion raw	Lecture CSV + movies_raw	3
F3 - Nettoyage	Logstash + normalisation	8
F4 - Mapping	Mapping explicite + analyzer	5
F5 - Requêtes	12 requêtes DSL commentées	5
F6 - Kibana	8 visualisations + dashboard	5
F7 - Docs	runbook, dictionnaire, nettoyage	3
F8 - Moteur	Mini moteur de recherche	5

8. Bilan, limites et améliorations

Points forts du projet

- Stack 100% reproductible en une commande sur une machine vierge avec Docker
- Pipeline Logstash complet : parsing, nettoyage, enrichissement et double indexation
- Mapping explicite soigné avec analyzer personnalisé adapté aux recherches cinéma
- 12 requêtes DSL couvrant tous les cas métier : recherche, filtrage, agrégation
- Dashboard Kibana avec 8 visualisations répondant à des questions métier concrètes
- Moteur de recherche full-text avec autocomplétion, fuzzy matching, 4 filtres combinables et synonymes
- Interface utilisateur soignée (design dark mode, animations, affichage enrichi par carte)
- Organisation projet professionnelle : Gitflow strict, PR/reviews, planning poker
- Script de démo (docs/demo_script.md) et GIF (docs/demo.gif) livrés pour la restitution

Points bloquants rencontrés

- Le volume du dataset (~700k lignes, 373 Mo) a nécessité une configuration mémoire Docker spécifique (4 Go minimum) pour éviter les crashes Logstash
- La gestion des champs multi-valués avec '-' comme séparateur a posé des problèmes lorsque des noms d'acteurs contenaient eux-mêmes des tirets
- Le syncedb Logstash a dû être supprimé manuellement pour forcer la réingestion lors des tests de pipeline
- Le mapping automatique d'Elasticsearch a nécessité la création d'un index v2 avec mapping explicite après constat des incohérences de types

Axes d'amélioration

- Ajouter un analyzer multilingue pour mieux traiter les films non anglophones
- Implémenter une recherche par similarité (more_like_this) dans le moteur de recherche
- Mettre en place des alertes Kibana sur les anomalies de qualité des données
- Versionner les pipelines Logstash avec des tests unitaires
- Enrichir le dataset avec des données complémentaires (box-office, streaming, awards)
- Ajouter une pagination et un scoring configurable dans le moteur de recherche